Introducción

*En proyectos que **integran inteligencia artificial**, el impacto de los costos operativos y la performance es determinante para la viabilidad y éxito de la solución, ya que un mal balance puede generar sobrecostos inesperados o por el contrario, degradar la experiencia del usuario.*

En esta clase vamos a aprender a **calcular y controlar los costos asociados al uso de modelos de IA**, entendiendo cómo factores como el consumo de tokens, la frecuenciade consultas y la configuración de los prompts afectan el presupuesto.

También exploraremos los diferentes **esquemas de uso y modelos de negocio** disponibles, para poder elegir la opción más adecuada según el contexto del proyecto. Finalmente, veremos estrategias prácticas para optimizar el rendimiento, reduciendo latencia y consumo sin sacrificar la calidad de las respuestas. De este modo, podrás tomar decisiones informadas que permitan desarrollar automatizaciones eficientes, sostenibles y escalables, tanto en entornos de experimentación como en producción.

Objetivos de la lecture 🚀

1. Identificar los factores que determinan los costos en workflows con IA (tokens, ejecuciones y almacenamiento).
2. Calcular el costo real de un modelo de IA usando métricas clave y escenarios comparativos.
3. Comparar modelos y plataformas de IA para elegir la opción más eficiente según el proyecto.
4. Aplicar estrategias de optimización en n8n para reducir costos y mejorar performance.

Estructura de Costos y Métricas Clave en Workflows con LLMs

Si cada llamada de IA cuesta apenas unos centavos, ¿por qué el presupuesto de un proyecto puede dispararse tan rápidamente? Basta con revisar el resumen de gastos de una semana de pruebas para notar que pequeñas acciones, multiplicadas por cientos o miles de ejecuciones, se transforman en una pendiente resbaladiza para cualquier equipo.

Hay tres componentes que marcan la diferencia en cualquier workflow que use modelos de lenguaje: **tokens**, **ejecuciones** y **almacenamiento**. Entender cómo se

combinan y cómo se miden es el punto de partida para cualquier decisión informada en automatización con IA.

Conceptos clave de costo: tokens, ejecuciones y almacenamiento

Cada vez que un workflow recurre a un LLM, el modelo "consume" texto en unidades llamadas ***tokens***. Un token no es necesariamente una palabra: puede ser una sílaba, una raíz verbal o incluso un fragmento de puntuación, dependiendo del idioma y del modelo. Por ejemplo, la frase *"Hola mundo!"* usualmente cuenta como tres tokens en GPT-4.

Los tokens se dividen en dos tipos:

Tokens de entrada	Tokens de salida
Corresponden al texto que se envía al modelo (el prompt, las instrucciones y, en algunos casos, el contexto previo).	Son los que el modelo genera como respuesta.

Los proveedores **cobran por tokens "procesados"** (entrada y salida juntas), así que una sola iteración puede sumar varios cientos o miles de ellos.

Por ejemplo, si el prompt ocupa 200 tokens y la respuesta generada tiene 300, el consumo total de la llamada será de 500 tokens procesados. Este número se usa para calcular el costo facturable.
Si el proveedor cobra USD 0,03 por cada 1.000 tokens, una llamada que procese 500 tokens tendrá un costo de (500 / 1.000) × 0,03 = **USD 0,015**.

¿La consecuencia? Una pregunta mal optimizada o un contexto de conversación innecesariamente largo puede duplicar o triplicar el costo de cada ejecución sin aportar valor real..

Pero los tokens por sí solos no explican todo. El **número de ejecuciones**, es decir, cuántas veces tu workflow invoca un modelo IA en un periodo dado, suele ser el factor de escalamiento invisible. Un trigger sencillo en n8n o Zapier que se dispare10 veces por hora, 24 horas al día, suma 7.200 llamadas al mes (30 días).

Cada llamada implica costos acumulativos en tokens, tiempos de respuesta y almacenamiento. Por eso, el control de la frecuencia de ejecución es tan importante como la optimización del contenido del prompt.

Finalmente, **el almacenamiento** se refiere a los costos por guardar resultados, logs o historiales de las interacciones IA en tu base de datos o sistema de archivos (ya sea en la nube o local). Si bien este componente muchas veces es menor a corto plazo, puede volverse relevante cuando se almacenan respuestas de modelos grandes, archivos adjuntos o historiales de cientos de megabytes.

Este costo no depende del modelo de IA, sino del proveedor de almacenamiento (por ejemplo, AWS S3, Google Cloud Storage o bases de datos como Firestore o Supabase), que cobran según el espacio ocupado (en GB/mes) y la frecuencia de acceso.

🎬 **Observa el siguiente video**

¿Qué métricas realmente importan?

Cuantificar el costo de operar IA va más allá del precio por 1.000 tokens. Algunas métricas son decisivas en el día a día:

- **Tokens por llamada a API:** Suma de tokens en el prompt y la respuesta. Influye directamente en el monto facturado.
- **Frecuencia de ejecuciones:** Cuántas veces, en promedio, se activa el workflow usando el LLM. Es un factor determinante en el costo total mensual.
- **Latencia:** Tiempo promedio que transcurre entre el momento en que se envía la solicitud (llamada a la API) y el momento en que se recibe la respuesta del modelo. No afecta el costo por tokens, pero sí impacta la eficiencia operativa, la experiencia del usuario y los costos de infraestructura.
    - Una latencia baja (por ejemplo, menos de 2 segundos) indica una respuesta rápida y mejora la experiencia del usuario.
    - Por el contrario, una latencia alta puede deberse a factores como la complejidad del prompt, la longitud del contexto, la carga del servidor o la calidad de la conexión de red.
    - Aunque no incrementa el costo directo del modelo, una mayor latencia implica un mayor uso de recursos computacionales, lo que puede elevar los costos de infraestructura (CPU, GPU, tiempo de espera en colas, etc.) y reducir la productividad en procesos dependientes del tiempo real.
- **Almacenamiento utilizado:** Espacio ocupado por logs, respuestas, o archivos generados, que puede generar cargos adicionales.

Controlar y proyectar estos valores ofrece claridad para anticipar picos de gasto y tomar decisiones proactivas, como ajustar prompts, reducir ejecuciones o limpiar datos almacenados innecesarios

---

¿Cómo se calcula el costo real? Del token al dólar

Ponerlo en números es más sencillo de lo que parece siempre y cuando se tengan claras las suposiciones.

Por ejemplo:

Un workflow automatiza respuestas inteligentes a tickets de soporte. Cada respuesta, sumando contexto precedente y salida del modelo, utiliza 500 tokens. La operación típica requiere manejar 100 tickets al día. El modelo elegido es GPT-4, tarifado en $0.03 por cada 1,000 tokens.

El costo diario de tokens sería lo puedes calcular desde un Notebook de Google Colab con el siguiente script (tal como este) :

TSPY

ExplainCopy

```
# Calculadora de costo diario de tokens


llamadas_dia = 100

tokens_por_llamada = 500

precio_por_1k_tokens = 0.03


# Costo por llamada

costo_x_llamada = (tokens_por_llamada / 1000) * precio_por_1k_tokens  # 0.015


# Costo diario

costo_diario = llamadas_dia * costo_x_llamada  # 1.50


# Costo mensual (30 días)

costo_mensual = costo_diario * 30  # 45.00
```

Por supuesto, si el flujo requiere guardar un log completo de cada interacción y, digamos, 1 MB por respuesta, el costo de almacenamiento también debe sumarse al total (especialmente en bases cloud con tarifas por GB/mes). Este gasto puede provenir de un servicio externo (como una base de datos o bucket en la nube), no del uso del modelo de IA en sí.

🤔 **Para Pensar:**
¿El costo más alto en el workflow viene realmente del modelo IA, o del volumen de

ejecuciones y el tamaño del contexto? Un cálculo erróneo en la estimación de tokens puede representar hasta un 100% de sobrecoste.

Ejemplo Práctico

Imagina este escenario sencillo para aplicar el razonamiento:

- Workflow A: 200 llamadas por día
- Tokens promedio/call: 700 tokens
- Modelo base: GPT-4, $0.03/1K tokens
- Días al mes: 30

¿Qué total mensual esperarías solo por tokens? Justifica el número y compara qué pasaría si se recorta el prompt a 400 tokens/call.

💡 **Solución:**

TSPY

ExplainCopy

(200 llamadas/día * 700 tokens/llamada) = 140,000 tokens/día

(140.000 tokens/día) * $0.03 / 1000 tokens = $4.20 diarios

$4.20 diarios * 30 dias = $126 mensuales

Si se hace una optimización el workflow y ahora solo se usan 400 tokens/call:

(200 llamadas/día * 400/llamada) = 80,000 tokens/día

(80.000 tokens/día) * $0.03 / 1000 tokens = $2.40 diarios

$2.40 diarios * 30 días = $72 mensuales

La diferencia por ajustar el prompt: $54 menos de consumo, solo optimizando contexto.

Incluso con un flujo relativamente simple, el impacto de cada ajuste se traduce rápidamente en ahorro tangible.

La elección del modelo de IA parece ser siempre el factor clave para el costo, pero el verdadero punto de quiebre suele estar en las ***decisiones de arquitectura y uso****: cuántas veces, con qué contexto, y qué hacer con los resultados. El siguiente paso es desentrañar cómo distintos modelos y plataformas comparan en costo y performance cuando se ponen a prueba con casos de uso reales.*

Tool extra

Te compartimos una calculadora de consumo de token, que no solo te hace un estimado de costos en tokens y en precio en USD y te recomienda el modelo más económico.

Benchmarking: Comparativa de Modelos, Plataformas y Estrategias de Uso

¿Siempre lo caro es mejor? Imagenemos que tienes que elegir entre dos plataformas para automatizar tareas con IA: una cuesta siete veces más por consulta que la otra, pero promete mejor rendimiento y resultados. ¿Cuál conviene a tu proyecto, y en qué situaciones el costo es una trampa o una inversión?

La respuesta no está solo en la etiqueta del precio. Detrás de cada modelo y plataforma, hay una combinación de factores determinan la eficiencia, la escalabilidad y la sostenibilidad de una solución de IA. Elegir bien requiere comparar más allá del costo por token, evaluando variables técnicas, de soporte y de control operativo.

Modelos de uso: de Free Tier a Open Source

Existen cuatro grandes modelos de uso cuando se trata de plataformas de IA y LLMs (Large Language Models). Cada uno presenta ventajas específicas y limitaciones que impactan costos, flexibilidad y riesgo operativo:

Free Tier	Pay-as-you-go
Ideal para pruebas y prototipos. Ofrece un número limitado de llamadas o tokens gratuitos por mes. El gran riesgo es alcanzar rápidamente el límite, quedarte sin acceso "de repente" o experimentar una calidad inferior en momentos de alta demanda.	El favorito para crecer de forma gradual. Solo pagas por lo que usas: cada token, llamada o GB de almacenamiento suma. Es fácil controlar el gasto en etapas iniciales, pero puede escalar velozmente con workflows de alta frecuencia. Muy útil para startups o proyectos variables en fase de validación.
Enterprise	**Open Source**
Paga una suscripción alta (mensual o anual) a cambio de soporte fuerte, SLA garatizados (Service Level Agreement),, funcionalidades premium y acceso a features avanzados, niveles superiores de seguridad y cumplimiento normativo. Clave para equipos grandes o exigencias críticas de estabilidad y compliance.	No tiene costo por uso del modelo, pero el usuario debe desplegar, configurar y mantener su propia infraestructura. Ofrece máxima flexibilidad, control sobre los datos y posibilidad de personalización, aunque exige conocimientos técnicos y tiempo de mantenimiento. Popular en organizaciones con restricciones legales, de privacidad o presupuestos muy ajustados.

El concepto de "**open source**" (código abierto) se refiere a software cuyo código fuente está disponible públicamente para que cualquiera pueda inspeccionarlo, modificarlo y distribuirlo. En el contexto de la IA, esto significa que el modelo o la herramienta puede

ejecutarse localmente o en servidores propios, sin depender de servicios externos. Sin embargo, la libertad de uso implica autogestionar recursos como CPU, RAM y almacenamiento, así como aplicar actualizaciones y garantizar la seguridad del entorno.

Un **SLA (Acuerdo de Nivel de Servicio, en inglés Service Level Agreement)** es un compromiso formal entre el proveedor y el cliente que define métricas de rendimiento y disponibilidad del servicio. Por ejemplo, un tiempo de actividad mínimo del 99.9%, tiempos máximos de respuesta ante incidencias o niveles de soporte técnico garantizados.

Estas categorías no son excluyentes. Muchos proyectos combinan, por ejemplo, un Free Tier para pruebas rápidas y Pay-as-you-go para producción inicial, migrando a Enterprise o Open Source solo si el crecimiento o estrategia lo justifica.

Plataformas y modelos

La teoría convence poco sin datos concretos para comparar. Imagina que necesitas automatizar la moderación de comentarios en tu web, procesando unas 10.000 consultas mensuales. Debes determinar cuál es el mejor modelo disponible que se adapta mejor a tu implementación, para ello deberias hacer una tabla comparativa en base a tus necesidades y
requerimientos.

Métrica / Plataforma	GPT-4 (Pay-as-you-go)	Claude (Free Tier)	Claude (Pay-as-you-go)	OLLAMA (Open Source)
Costo por 1K tokens	$0,03	Gratis (limitado)	$0,01	0 (infraestructura propia)
Límite mensual (Gratuito)	No Aplica	10.000 tokens aprox.	No Aplica	No Aplica
Latencia promedio	5-10 s	8-15 s	5-10 s	2-8 s *
Soporte y SLA	Básico	Nulo	Básico	Depende del despliegue
Facilidad de integración	Alta	Media	Alta	Baja - requiere setup
Escalabilidad	Muy alta	Limitada	Muy alta	Depende de los recursos

** Ollama es una plataforma open source que se ejecuta localmente en tu equipo o servidor. Su latencia depende directamente del hardware donde se ejecute, ya que el procesamiento ocurre de forma interna, sin depender de servidores externos. En consecuencia, no tiene costo por token, pero consume CPU, GPU y RAM propias. Cuanto más potente sea el hardware, menor será la latencia y mayor la capacidad de procesar consultas simultáneas.*

A diferencia de los modelos SaaS (Software as a Service), donde el procesamiento ocurre en la nube y el usuario paga por token o uso, Ollama traslada el costo económico hacia la infraestructura física, lo que puede resultar conveniente para proyectos con alta demanda pero recursos computacionales disponibles.
En la práctica, Claude Free Tier puede parecer imbatible para un piloto chico: costo

cero y resultados razonables Sin embargo, rápidamente se vuelve inadecuado si el flujo crece o necesitas SLA concretos. GPT-4 sobresale en calidad, pero cada llamada "premium" se paga. OLLAMA, por su parte, elimina el gasto variable, pero exige saber conocimientos técnicos, operar infraestructura y aceptar mantenimiento propio.

🤔 ***Para Pensar:*** *¿Por qué algunos equipos migran de soluciones pagas a open source tras pasar la etapa de validación? ¿Qué riesgos puede implicar no contar con soporte o SLA?*

Los modelos LLM son solo una parte. El ecosistema de automatización suma capas: plataformas como n8n, Make, y otros agregan diferencias en integración, coste operativo, velocidad y facilidad de mantenimiento.

Caso rápido: ¿Claude o GPT-4 para 10,000 consultas/mes?

Supongamos que el flujo emplea un prompt promedio de 500 tokens por llamada.

- **GPT-4**: 10.000 * 500 tokens = 5,000,000 tokens ($0.03 x 5,000 = $150/mes)
- **Claude Pay-as-you-go**: 10.000 * 500 = 5,000,000 tokens ($0.01 x 5,000 = $50/mes)
- **Claude Free**: Solo soporta hasta 10,000 tokens gratuitos, es decir, apenas cubre 20 llamadas. Práctico solo para prototipos.
- **OLLAMA Open Source**: Cero costo variable, pero exige instalar y mantener el modelo en servidores propios, consumiendo recursos locales.

En experiencia real, equipos inician con Free Tier para testear ideas, pasan a Pay-as-you-go en prototipos productivos, y evaluan opciones Enterprise u Open Source solo cuando consolidan el crecimiento, mayor control, la necesidad de cumplimiento regulatorio o la previsibilidad financiera.

Criterios estratégicos para elegir la mejor herramienta

Elegir una plataforma no es solo buscar el menor costo por token o velocidad. Hay que cruzar **criterios técnicos, estrategicos y económicos**, evaluando siempre el contexto y el estado de madurez del proyecto:

👉 Factores técnicos críticos

- **Rendimiento y latencia**: Clave si la experiencia del usuario es sensible a la espera.
- **Calidad de resultados**: No todos los modelos cumplen igual ante los matices de idioma, contexto, o formatos. Algunos modelos se desempeñan mejor para ciertos contextos o tareas (ejemplo: generación de código, textos largos, validaciones).
- **Escalabilidad**: Capacidad de la plataforma para sostener incrementos de tráfico. ¿La plataforma resiste si tu tráfico crece 10x o 100x?

- **Facilidad de integración y soporte con tus otras herramientas**: ¿Hay conectores listos, APIs o necesitas desarrollo propio?
- **SLA y soporte**: Nivel de garantía ante fallos o interrupciones del servicio. ¿Qué pasa si el servicio falla un viernes a las 22h?

👉 **Variables económicas**

- **Costo total de propiedad (TCO)**: Suma no solo el precio de tokens/llamadas, sino mantenimiento, infraestructura y tiempo de operación.
- **Previsibilidad de gasto**: Un sistema "barato" pero impredecible puede ser más riesgoso que uno caro pero plano y controlable.

👉 **Estrategia según fase de proyecto**

- **Piloto:** Free Tier o herramientas rápidas de integrar, priorizando la velocidad.
- **Escalamiento:** Pay-as-you-go con control de costos y métricas, con previsibilidad financiera y menos riesgos de caídas o suspensión.
- **Producción estable:** Enterprise o Open Source, priorizando seguridad, autonomía y continuidad.

💡 ***Ejemplo:***
Un startup de edtech lanza un chatbot usando GPT-4 (Pay-as-you-go a $0.03/1K tokens). Rápidamente, el crecimiento de usuarios eleva los costes mensuales de $30 a $400 USD. Evaluan migrar a Claude ($120 USD/mes) o instalar OLLAMA en servidores de su universidad (costo casi nulo, pero un ingeniero dedica 8h/mes a mantenimiento).

La decisión óptima combina el volumen, la capacidad de operar infraestructura y la calidad percibida por sus usuarios. En ocasiones, invertir en infraestructura propia es más rentable a largo plazo que pagar por uso, siempre que el equipo tenga las competencias necesarias para gestionarla.

Recomendaciones Prácticas: Estrategias de Optimización y Buenas Prácticas en n8n

Reducir el costo de un workflow que usa modelos de IA un 20% sin afectar la calidad parece un desafío solo para grandes empresas... hasta que descubres que muchas de las mejores optimizaciones son simples ajustes técnicos y decisiones inteligentes

dentro de herramientas como n8n. El reto está en **diseñar flujos tan eficientes como robustos, sin sacrificar resultados**.

El corazón del ahorro: optimizar cada paso

Un workflow IA típico en n8n conecta un trigger (un evento o input) con un nodo de LLM (modelo de lenguaje) y, finalmente, con una acción que consume o presenta la respuesta (un email, registro, alerta, etc.). Cada uno de estos pasos puede impactar tanto en **performance** (velocidad, capacidad de respuesta) como en **costos directos** (tokens procesados, llamadas facturadas).

Optimizar implica reducir redundancias, reutilizar resultados y minimizar la cantidad de información enviada al modelo, sin perder calidad.

Procesamiento por lotes: más resultados, menos llamadas

Procesar elementos de uno en uno siempre parece lo más fácil, pero cuando una tarea se repite decenas o cientos de veces al día, el costo escala rápidamente.
El **procesamiento por lotes (batch processing)** consiste en agrupar varias entradas o tareas similares y procesarlas juntas en una sola ejecución del modelo. Esto reduce la cantidad de llamadas al modelo y optimiza el uso de tokens, porque el contexto se comparte entre ítems dentro del lote.

Caso práctico

Una empresa recibe 100 comentarios diarios de clientes para analizar sentimiento.

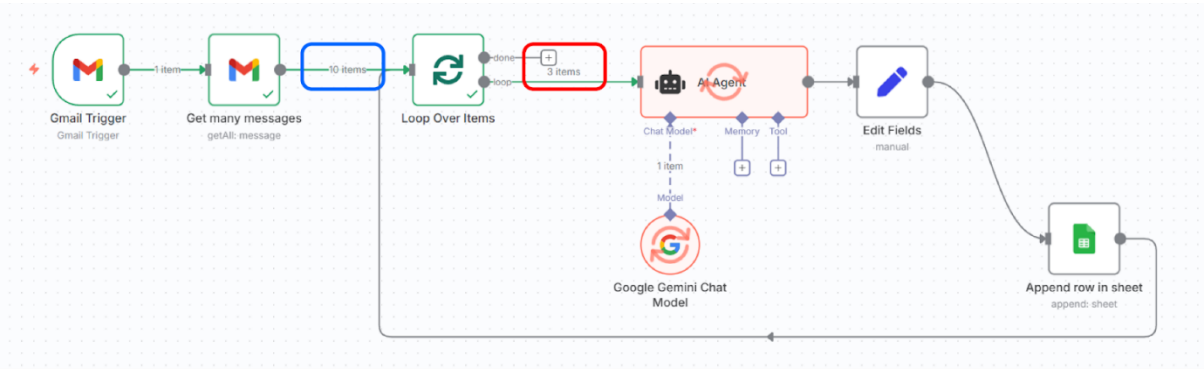
- **Sin lotes**: cada comentario se envía individualmente, por lo tanto son 1.000 llamadas al modelo.
- **Con lotes de 20**: los comentarios se agrupan y solo se realizan 50 llamadas totales.
  Además, el modelo reutiliza parte del contexto ("analiza sentimientos de estos 10 textos"), lo que puede reducir el número total de tokens necesarios por comentario.

Implementación en n8n

El nodo "SplitInBatches" permite dividir datos en grupos de tamaño fijo. Un flujo eficiente podría tener esta estructura:

1. Trigger: Entrada de datos (comentarios, correos, reseñas).
2. Se filtra los últimos 10 mensajes afines.
3. SplitInBatches: agrupa entradas de a 3.
4. Nodo LLM: analiza cada lote en una sola llamada.

5. Nodo de almacenamiento: guarda resultados (por ejemplo, en Google Sheets o una base SQL).



Una ventaja adicional es que este método **mejora la estabilidad del flujo y evita saturar el modelo** con demasiadas peticiones simultáneas.

Se recomienda evitar lotes demasiado grandes (por encima de 40 ítems o 8.000 tokens), ya que podrían generar ***timeouts*** o respuestas incompletas.

Uso de caché: no repitas lo que ya sabes

Cada llamada a un modelo tiene un costo. Un buen sistema evita preguntar dos veces lo mismo. Si una entrada ya fue procesada, almacenar la respuesta (caché) evita consumir de nuevo los recursos del modelo. Esto es especialmente efectivo en workflows que enfrentan inputs repetidos o levemente distintos pero equivalentes.

Ejemplo práctico

En la moderación de publicaciones, ciertos mensajes recurrentes ("¡Hola mundo!", saludos) pueden ser cacheados y su resultado reutilizado. En n8n, puedes implementar esto así:

1. Agrega un nodo “IF” o “HTTP Request” para consultar si la entrada ya existe en una base local (puede ser un Google Sheet, una tabla de Supabase, Redis o incluso un archivo JSON).
2. Si existe, el flujo salta directamente al paso final (usa el resultado almacenado).
3. Si no existe, envía la entrada al LLM y luego guarda el resultado en el caché.

Para implementar caché en n8n hay distintas opciones:

- **Opción simple**: usar Google Sheets con una columna “input” y otra “output”. Antes de enviar un nuevo texto, n8n busca si ya existe.
- **Opción avanzada**: usar Redis o una base SQL local para búsquedas más rápidas y almacenamiento temporal.

Este patrón puede reducir entre un 15% y 40% del costo mensual en flujos con entradas recurrentes.

🎬 **Observa el siguiente video**

Prompts concisos: menos es más

Cada token que se envía al modelo cuenta para el costo y también afecta el tiempo de respuesta. A menudo, los prompts crecen por acumulación de instrucciones o ejemplos. Redactar prompts compactos y claros ahorra tokens, pero también ayuda a la consistencia de las respuestas.

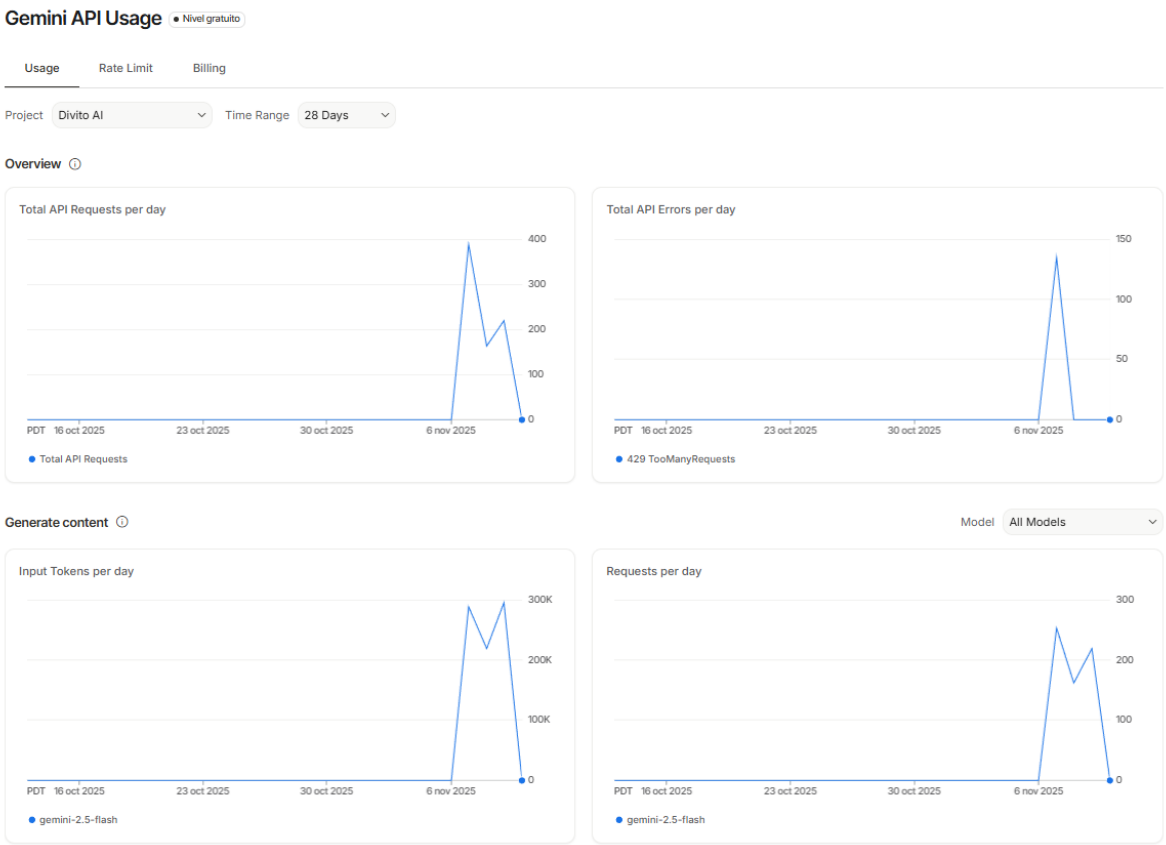
Comparación real:

- Prompt extenso:

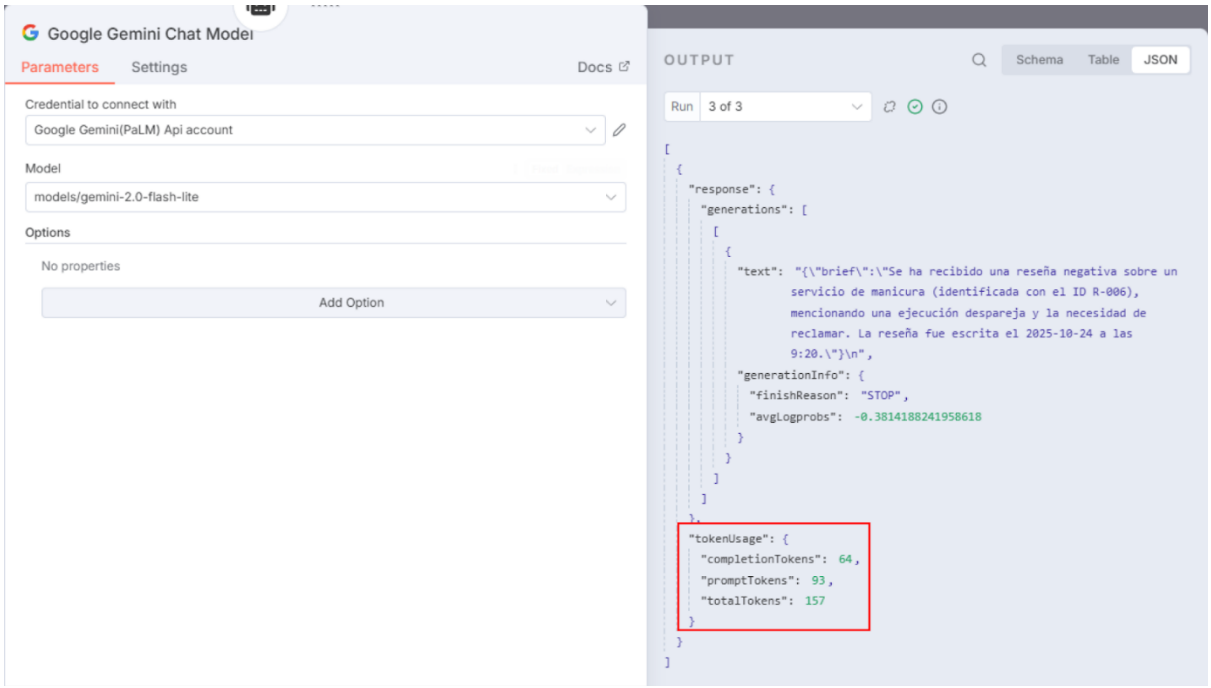
Eres un asistente que analiza mensajes para moderar contenido ofensivo. Considera las reglas A, B, C, D, E... [Mensaje completo]

- Prompt optimizado:

¿El texto contiene lenguaje ofensivo? Sí/No. [Mensaje completo]
Al revisar los logs de n8n, ***reducir el prompt puede significar un ahorro de hasta 30% en tokens****, especialmente importante en flujos de alta frecuencia. En la mayoría de las integraciones con modelos de IA (OpenAI, Anthropic, etc.), los datos de uso se registran en el log de la API.*
Al revisar los logs de n8n, reducir el prompt ***puede significar un ahorro*** *de hasta 30% en tokens, especialmente importante en flujos de alta frecuencia. En la mayoría de las integraciones con modelos de IA (OpenAI, Anthropic, etc.), los datos de uso se registran en el log de la API.*



En n8n, la mayoría de los nodos de LLM (como OpenAI o Anthropic) devuelven información del uso de tokens en la salida JSON de cada ejecución, normalmente dentro de un campo como **tokenUsage.total_tokens** o similar.



Reutilizar contexto: solo transmite lo necesario

Si en un workflow **ya se conoce parte del contexto relevante** (por ejemplo, reglas de negocio o información de referencia), **no es necesario repetirla en cada llamada**. Guardar estos datos en variables o tablas que sólo se consultan cuando cambia la regla evita gastos innecesarios.

Herramientas para reutilizar contexto:

Puedes usar **Redis** para guardar información temporal (como configuraciones o datos de referencia) y consultarla rápidamente cuando se necesite. Esto evita la repetición de consultas costosas o innecesarias.

Si no usas Redis, puedes almacenar datos de contexto en una **base de datos SQL** o en **Google Sheets**. Estas opciones permiten guardar información persistente y consultarla cuando sea necesario.

Si la información solo se necesita dentro del mismo flujo, puedes usar **variables internas** de n8n para guardarla y reutilizarla.

Ejemplo práctico:

Imagina que en tu flujo tienes que consultar si un cliente tiene un descuento especial. En lugar de consultar una base de datos cada vez que se necesita, guardas esa información en Redis al inicio del flujo. Así, en los pasos siguientes, solo consultas Redis, evitando tener que consultar la base de datos constantemente, lo cual hace el flujo más rápido.

👍 Buenas prácticas clave para cualquier workflow de IA en n8n

- Empieza por medir: Sin línea base, no hay optimización real
- Optimiza uno o dos factores por vez y evalúa impacto individual
- Prefiere batches razonables (no demasiado grandes: riesgo de timeouts)
- Si cacheas, define vigencia y lógica de actualización para evitar respuestas antiguas o no relevantes
- Redacta prompts mínimos pero claros; revisa ejemplos y elimina lo accesorio
- Documenta cada cambio con un changelog: facilita auditoría y transferir mejoras a otros flujos

---

De la optimización técnica al control estratégico

A medida que los workflows escalan, la optimización puntual no basta. Es necesario definir reglas operativas para limitar costos y prevenir errores inesperados. El siguiente paso es **transformar estas lecciones técnicas en políticas automáticas y checklist de control**, asegurando que los ahorros logrados no se pierdan en el futuro.

🎬 **Observa el siguiente video**

Te mostramos en tiempo real cómo un flujo simple en n8n puede optimizarse aplicando técnicas vistas en la clase (batching, prompts más cortos y caché), evidenciando el impacto cuantitativo en tokens, costo y latencia.

Previous

Cierre

En esta lecture viste que **el costo de usar IA** no es un "detalle financiero" que se revisa al final, sino **una pieza central del diseño de cualquier workflow**. Aprendiste a desarmar ese costo en partes concretas: tokens, cantidad de ejecuciones y almacenamiento, y a traducir esos números en montos diarios y mensuales que sí puedes gestionar. También viste cómo **métricas** como latencia, frecuencia de llamadas y uso de almacenamiento se conectan con la experiencia del usuario y con la salud de tu infraestructura.

Además, exploraste **distintos modelos de uso y plataformas** (Free Tier, Pay-as-you-go, Enterprise y Open Source), entendiendo que **no hay una única respuesta correcta, sino decisiones estratégicas** según la etapa del proyecto, el volumen esperado y la capacidad de tu equipo. Analizaste casos comparativos entre modelos como GPT-4, Claude u opciones open source como OLLAMA, para ver que "más caro" no siempre significa "mejor", y que a veces la verdadera diferencia está en el control operativo y los SLA.

Finalmente, aterrizaste todo en **prácticas concretas dentro de n8n**: procesar por lotes para reducir llamadas, usar caché para no pagar dos veces por la misma respuesta, redactar prompts más concisos y reutilizar contexto solo cuando aporta valor. Viste que **pequeños ajustes técnicos pueden representar ahorros significativos sin sacrificar calidad**.

Quédate con esta idea:

Dominar costos y performance no es un freno a la innovación, sino el habilitador para que tus automatizaciones con IA sean sostenibles, escalables y defendibles frente a cualquier stakeholder. Estás construyendo la base para diseñar sistemas con IA que no solo funcionen bien, sino que también cierren los números.